

The Two-Timescale Bayesian Update Problem

A Complete Mathematical Reference

1. The Core Tension

In any probabilistic model with hyperparameters, there is a fundamental asymmetry between two kinds of unknowns. **Function values** — the actual outputs of the unknown function at specific inputs — are directly observable, one new data point at a time. Each observation immediately and precisely constrains the function value in its neighborhood. **Hyperparameters** — the length scale, the noise variance, the coregionalization matrix — describe the *global statistical structure* of the function. No single observation provides much information about global structure; you need to accumulate many observations spread across the input space, observe the pattern they form, and infer the structural properties from that pattern.

This asymmetry in the timescale of informational value is the two-timescale problem. Treating both unknowns as if they update at the same rate is wasteful at best and misleading at worst. The appropriate response is to use an inference algorithm that explicitly operates on two different timescales: fast updates to the function posterior given fixed hyperparameters, and slow updates to the hyperparameters as evidence accumulates.

2. The Hierarchical Bayesian Formulation

The two-timescale problem has a clean formulation as a **hierarchical Bayesian model**. The complete generative model is:

$$\begin{aligned} & p(\theta) \sim p(\theta) \quad \text{(prior over hyperparameters)} \\ & f \mid \theta \sim \mathcal{GP}(0, k_\theta) \quad \text{(function drawn from GP with kernel parametrized by } \theta \text{)} \\ & y_i \mid f, x_i \sim \mathcal{N}(f(x_i), \sigma_n^2) \quad \text{(noisy observations)} \end{aligned}$$

A fully Bayesian treatment would maintain a joint posterior over both f and θ simultaneously:

$$p(f, \theta \mid \mathbf{y}) \propto p(\mathbf{y} \mid f), p(f \mid \theta), p(\theta)$$

This is theoretically correct but computationally intractable for most kernels, because the posterior over θ doesn't have a closed form and marginalizing over f to get $p(\theta \mid \mathbf{y})$ requires evaluating an integral that has no closed-form solution.

3. The Empirical Bayes Approximation

The standard approximation used in virtually all practical GP implementations is **Empirical Bayes** (also called Type-II Maximum Likelihood). Rather than maintaining a full posterior over θ , you find a point estimate $\hat{\theta}$ that maximizes the **marginal likelihood** — the probability of the data integrated over all possible function values:

$$p(\mathbf{y} \mid X, \theta) = \int p(\mathbf{y} \mid f), p(f \mid \theta), df$$

For a GP with Gaussian noise, this integral is analytically tractable and equals:

$$\log p(\mathbf{y} \mid X, \theta) = -\frac{1}{2} \mathbf{y}^T K_{\theta}^{-1} \mathbf{y} - \frac{1}{2} \log |K_{\theta}| - \frac{n}{2} \log(2\pi)$$

Maximizing this over θ gives $\hat{\theta}$, which is then treated as if it were the true hyperparameter for all subsequent posterior inference. This is the two-timescale approximation: on the fast timescale, update the function posterior with $\hat{\theta}$ held fixed; on the slow timescale, periodically recompute $\hat{\theta}$ by re-maximizing the marginal likelihood on all accumulated data.

The name “Empirical Bayes” reflects the fact that the prior $p(\theta)$ is effectively being replaced by a point mass at $\hat{\theta}$ — the prior is “estimated from the data” rather than specified in advance, which is a conceptual departure from orthodox Bayesian inference but a practical necessity for tractability.

4. Stochastic Approximation and the Robbins-Monro Framework

The convergence behavior of two-timescale algorithms is characterized by the **Robbins-Monro framework** for stochastic approximation, developed in 1951 and since generalized extensively.

The basic Robbins-Monro algorithm seeks the zero of a function $h(\theta)$ by iterating:

$$\theta_{t+1} = \theta_t - \alpha_t \hat{h}(\theta_t)$$

where $\hat{h}(\theta_t)$ is a noisy estimate of $h(\theta_t)$ and α_t is a step size

sequence. The conditions for almost-sure convergence to the true zero θ^* are:

$\sum_{t=1}^{\infty} \alpha_t = \infty$ (steps are large enough to reach the solution)
 $\sum_{t=1}^{\infty} \alpha_t^2 < \infty$ (steps are small enough to average out noise)

The canonical choice satisfying both conditions is $\alpha_t = c/t^\rho$ for some constant $c > 0$ and exponent $\rho \in (0.5, 1]$.

Applied to two-timescale hyperparameter optimization, the “step size” corresponds to the frequency of re-optimization. Re-optimizing at every observation corresponds to $\rho \rightarrow 0$ (constant step size) — this doesn’t converge but tracks a slowly drifting optimum well. Reducing the optimization frequency as t grows (larger ρ) converges to the true hyperparameters in a stationary environment but adapts slowly when the environment changes.

5. The Logarithmic Optimization Schedule

A practical schedule that balances adaptation speed with computational cost is the **logarithmic schedule**: perform hyperparameter optimization at total observation counts $n_0, 2n_0, 4n_0, 8n_0, \dots$

The rationale is information-theoretic. Early observations are maximally informative about global structure — each one reveals something qualitatively new about the function’s behavior. The first 10 observations might reveal whether the function is monotone, whether it has a peak, approximately where the length scale is. The next 10 observations refine these estimates but don’t change the qualitative picture as dramatically. By observation 1000, additional data provides only marginal improvement to hyperparameter estimates.

The logarithmic schedule reflects this diminishing return: you optimize frequently early (when marginal value is high) and infrequently late (when marginal value is low). The computational cost per optimization run is $O(n^3)$, but the total cost over all T observations is $O(T^3 / \log T)$, since you run $O(\log T)$ optimization calls at costs $O(n_0^3), O((2n_0)^3), \dots, O(T^3)$ — dominated by the last one.

In code, the schedule is detected by checking whether the current observation count is a power of two (after an initial offset):

```
def should_optimize(n_obs: int, base: int = 5) -> bool:
    # Check if (n_obs - base) is a power of 2
    # Powers of 2 satisfy: n & (n - 1) == 0
    n = n_obs - base
    return n > 0 and (n & (n - 1)) == 0
```

6. The Feedback Loop and Overfitting Risk

The two-timescale approximation has a subtle failure mode that is important to understand. When you optimize $\hat{\theta}$ by maximizing $\log p(\mathbf{y} \mid X, \hat{\theta})$, you are fitting hyperparameters to the same data that your function estimates are based on. This creates a **feedback loop**: the hyperparameters are tuned to explain the current observations, the posterior is computed under those hyperparameters, the next action is chosen based on that posterior, and the resulting observation is used to further tune the hyperparameters.

When data is abundant, this feedback loop is benign — the marginal likelihood is a consistent estimator of the true hyperparameters, and the feedback loop converges to the truth. When data is scarce, the loop can produce **overfitting in hyperparameter space**: the optimizer finds a length scale that perfectly explains the few available observations but would generalize poorly to new inputs.

The principled solution is to add a **prior on hyperparameters** and maximize the *posterior* over hyperparameters rather than the likelihood:

$$\hat{\theta} = \arg\max_{\theta} \log p(\mathbf{y} \mid X, \theta) + \log p(\theta)$$

This is **MAP hyperparameter estimation**. Common choices include a log-normal prior on the length scale (penalizing both very small and very large values) and a half-normal prior on the noise variance (penalizing large noise). These priors implement a soft version of the coverage-based regularization discussed for the coregionalization matrix.

7. The Two Timescales in Practice: Fast and Slow Paths

A clean implementation separates the fast and slow timescales into distinct code paths with distinct concurrency responsibilities.

The **fast path** runs in the main decision-making loop. At each iteration it: takes a snapshot of current hyperparameters under a brief lock acquisition, computes the GP posterior under those hyperparameters (expensive, but lock-free), draws a Thompson sample, executes the selected action, observes a reward, and appends the new observation to the data store. The key invariant is that the fast path *never modifies hyperparameters* — it only reads them.

The **slow path** runs in a background thread. It waits for a signal that sufficient new data has arrived, snapshots the current observation data, runs the marginal likelihood optimization

(potentially expensive, fully lock-free on the snapshot), and deposits the result in a communication channel. The key invariant is that the slow path *never reads or writes application state directly* — it only reads observation data (via snapshot) and writes to the communication channel.

The two paths communicate through an **atomic hand-off**: the slow path deposits a new hyperparameter vector in a queue, and the fast path picks it up at the start of its next iteration. This hand-off acquires a lock only long enough to atomically replace the old hyperparameter vector with the new one. The atomicity is essential in the multi-output case, where length scale and coregionalization matrix must be updated together — updating one without the other would produce an inconsistent model.

8. Non-Stationarity and the Limits of Two-Timescale Convergence

The Robbins-Monro convergence results assume a **stationary** environment — the true hyperparameters don't change over time. When the environment is non-stationary, the two-timescale approach cannot converge in the classical sense, because the target θ^* is itself moving.

Instead, the appropriate notion of optimality is **tracking**: the hyperparameter estimates should follow the drifting true values closely enough that the model remains approximately well-specified. The tracking accuracy depends on the ratio of the optimization step size to the drift rate — faster drift requires more aggressive updates to maintain tracking.

For environments with known periodicity (like the periodic kernel application), a partially supervised approach is possible: the period p is constrained to a bounded range during optimization (reflected in the bounds on $\log p$ in the L-BFGS-B call), and the optimizer seeks the best period within that range rather than searching all of $\mathbb{R}^+$. This regularizes the optimization and prevents the period estimate from drifting to pathological values when evidence is weak.

See also: Gaussian Process Regression; Multi-Output GP and Coregionalization; Non-Stationary Gaussian Processes; Concurrency Design Patterns